

Glosario Java

Índice

- [Glosario Java — Piezas que se repiten en los ejercicios](#)
 - [1. Estructura básica de un programa](#)
 - [2. Modificadores de acceso y de comportamiento](#)
 - [3. Tipos de datos primitivos](#)
 - [4. Entrada y salida de datos](#)
 - [5. La clase Math](#)
 - [6. Operadores](#)
 - [7. Estructuras de decisión](#)
 - [8. Ciclos \(bucles\)](#)
 - [9. Métodos y clases \(Programación Orientada a Objetos\)](#)
 - [10. Arreglos y listas](#)
 - [Consejo para el examen](#)

Glosario Java — Piezas que se repiten en los ejercicios

Guía de referencia rápida. Cuando el ingeniero señale una línea, revisa aquí qué tipo de pieza es y qué función cumple en general.

1. Estructura básica de un programa

Pieza	Función general
<code>public class NombreClase { }</code>	Todo código Java vive dentro de una clase. Si es <code>public</code> , el archivo debe llamarse igual (<code>NombreClase.java</code>).
<code>public static void main(String[] args) { }</code>	Punto de entrada del programa. Java siempre busca este método exacto para empezar a ejecutar.
<code>//texto</code>	Comentario de una línea. Java lo ignora por completo, es solo una nota.
<code>/* texto */</code>	Comentario de varias líneas.
<code>;</code>	Termina cada instrucción (sentencia). Obligatorio.
<code>{ }</code>	Delimitan un bloque de código: el cuerpo de una clase, un método, un <code>if</code> , un ciclo, etc.

2. Modificadores de acceso y de comportamiento

Palabra	Función general
<code>public</code>	Accesible desde cualquier otra clase.
<code>private</code>	Solo accesible dentro de la misma clase. Se usa mucho en atributos, protegidos por <i>getters/setters</i> .
<code>static</code>	Pertenece a la clase entera, no a un objeto específico.

	Hay una sola copia compartida por todos. Se puede llamar sin crear un objeto.
<code>void</code>	El método no devuelve ningún valor.
<code>final</code>	El valor, una vez asignado, no se puede cambiar después. Se usa para constantes.

3. Tipos de datos primitivos

Tipo	Guarda
<code>int</code>	Números enteros.
<code>double</code>	Decimales de alta precisión (el más usado para decimales).
<code>float</code>	Decimales de menor precisión que <code>double</code> (ocupa menos memoria).
<code>long</code>	Enteros muy grandes.
<code>short</code>	Enteros de tamaño intermedio (entre <code>byte</code> e <code>int</code>).
<code>byte</code>	Enteros muy pequeños (-128 a 127), ahorra memoria.
<code>boolean</code>	Solo <code>true</code> o <code>false</code> .
<code>char</code>	Un solo carácter, siempre con comillas simples <code>'x'</code> .
<code>String</code>	Texto (cadena de caracteres). No es primitivo, es una clase — usa comillas dobles <code>"texto"</code> .

4. Entrada y salida de datos

Pieza	Función general
<code>System.out.println(...)</code>	Imprime en consola y salta de línea.
<code>System.out.printf(...)</code>	Imprime con formato controlado (usa marcadores como <code>%s</code> , <code>%d</code>).
<code>String.format(...)</code>	Igual que <code>printf</code> , pero en vez de imprimir, devuelve el texto ya formateado como <code>String</code> .
<code>%s</code>	Marcador de posición para un <code>String</code> .
<code>%d</code>	Marcador de posición para un entero.
<code>%02d</code>	Entero con mínimo 2 dígitos, rellena con <code>0</code> a la izquierda (ej. <code>9</code> → <code>09</code>).
<code>\n</code>	Carácter especial de salto de línea.
<code>Scanner x = new Scanner(System.in);</code>	Crea un objeto para leer datos que el usuario escribe por teclado.
<code>x.nextInt()</code>	Lee un número entero escrito por el usuario (pausa el programa hasta que lo escriban).

5. La clase Math

Método	Función
<code>Math.pow(base, exp)</code>	Potencia.
<code>Math.sqrt(n)</code>	Raíz cuadrada.
<code>Math.floor(n)</code>	Redondea hacia abajo.
<code>Math.ceil(n)</code>	Redondea hacia arriba.
<code>Math.round(n)</code>	Redondeo normal (matemático).
<code>Math.random()</code>	Número decimal aleatorio entre 0.0 y 1.0 (sin parámetros).

6. Operadores

Operador	Función
<code>=</code>	Asignación (guardar un valor en una variable). No es igualdad.
<code>==</code>	Comparación de igualdad.
<code>!=</code>	Comparación de “distinto de”.
<code>> , < , >= , <=</code>	Comparaciones de mayor/menor.
<code>&&</code>	AND lógico — ambas condiciones deben ser verdaderas.
<code>\ \ </code>	OR lógico — al menos una condición debe ser verdadera.
<code>+</code>	Suma (con números) o concatena texto (si hay un String involucrado; se evalúa de izquierda a derecha).
<code>++</code>	Incremento en 1 (<code>x++</code> es lo mismo que <code>x = x + 1</code>).
<code>+=</code>	Suma y asigna en un paso (<code>suma += n</code> es <code>suma = suma + n</code>).
<code>condición ? valorSiTrue : valorSiFalse</code>	Operador ternario — un <code>if/else</code> compacto en una sola línea.

7. Estructuras de decisión

Pieza	Función general
<code>if (condición) { }</code>	Ejecuta el bloque solo si la condición es verdadera.
<code>else if (condición) { }</code>	Se evalúa solo si el <code>if</code> anterior fue falso.
<code>else { }</code>	Se ejecuta si ninguna condición anterior se cumplió.
<code>switch(variable) { case valor: ... break; }</code>	Compara una variable contra varios valores posibles, de forma más ordenada que muchos <code>if/else</code> .
<code>break;</code>	Corta la ejecución del <code>switch</code> en ese punto (sin él, sigue ejecutando los casos siguientes — “fall-through”).
<code>default:</code>	Se ejecuta si ningún <code>case</code> coincidió.

8. Ciclos (bucles)

Ciclo	Función general	Detalle clave
<code>while (condición) { }</code>	Repite mientras la condición sea verdadera.	Revisa la condición antes de cada vuelta — puede no ejecutarse nunca.
<code>do { } while (condición);</code>	Igual que <code>while</code> , pero revisa la condición al final .	Se ejecuta al menos una vez , sin importar la condición.
<code>for (inicio; condición; incremento) { }</code>	Ideal cuando ya sabes cuántas veces repetir.	Junta inicialización, condición e incremento en una sola línea.
<code>for (Tipo variable : coleccion) { }</code>	“for-each” — recorre todos los elementos de un arreglo/lista sin usar índice manual.	Más simple, pero no da acceso directo a la posición (índice).

9. Métodos y clases (Programación Orientada a Objetos)

Pieza	Función general
<code>NombreClase nombreObjeto = new NombreClase(...);</code>	Crea (instancia) un objeto a partir de una clase.
<code>Constructor (public NombreClase(...) { })</code>	Método especial que se ejecuta automáticamente al crear un objeto con <code>new</code> . Mismo nombre que la clase, sin tipo de retorno.
Sobrecarga de constructores	Tener varias versiones del constructor con distinta cantidad/tipo de parámetros. Java elige cuál usar según cómo lo llames.
<code>this</code>	Se refiere al objeto actual — sirve para diferenciar un atributo de la clase de un parámetro con el mismo nombre.
<code>Setter (setAlgo(valor))</code>	Modifica un atributo privado desde fuera de la clase.
<code>Getter (getAlgo())</code>	Lee/devuelve el valor de un atributo privado.
<code>return valor;</code>	Devuelve un valor y termina la ejecución del método ahí mismo.
<code>toString()</code>	Define cómo se convierte un objeto a texto (por ejemplo al imprimirlo). Si no lo defines, Java usa un formato genérico poco útil.
<code>extends</code>	Herencia — una clase “hija” reutiliza atributos/métodos de una clase “padre”, sin reescribirlos.
Polimorfismo	Tratar objetos de distintas clases hijas de forma uniforme, a través de su clase padre en común (ej. un arreglo del tipo padre que guarda objetos de varias subclases).

10. Arreglos y listas

Pieza	Función general
<code>int arr[] = {1,2,3};</code>	Arreglo (colección de valores del mismo tipo, tamaño fijo).
<code>arr[0]</code>	Acceso por índice — siempre empieza en 0 .

<code>arr.length</code>	Cantidad de elementos del arreglo (propiedad, sin paréntesis).
<code>LinkedList<Tipo></code>	Lista dinámica (puede crecer/encoger), a diferencia del arreglo de tamaño fijo.
<code>.add(valor)</code>	Agrega un elemento al final de la lista.
<code>.addFirst(valor)</code>	Agrega un elemento al inicio de la lista.
Arreglo como parámetro de método	Se pasa por referencia — si el método lo modifica, el cambio se ve también fuera del método.

Consejo para el examen

Cuando el ingeniero señale una línea, hazte 3 preguntas rápidas: 1. **¿Qué tipo de pieza es?** (¿declaración de variable, estructura de control, llamada a método, constructor?) 2. **¿Qué hace exactamente esa línea, paso a paso?** 3. **¿Por qué está ahí / qué rol cumple en el programa completo?**

Responder con esas 3 capas suele ser justo lo que buscan que demuestres.